

CS 320 Project Two: Summary and Reflections Report

Cindi Rodriguez

Southern New Hampshire University

CS 320: Software Testing, Automation, and Quality Assurance

Jonathan Norman

August 16, 2026

CS 320: Project Two: Summary and Reflections Report

Summary

For Project One, I used JUnit 5 to test the contact, task, and appointment services. My main approach was to look at the requirements for each part of the project and create tests that checked both valid and invalid information. For the Contact class, I tested the contact ID, first and last name, phone number, and address. For example, lines 19-23 of `ContactTest.java` use `assertEquals` to make sure a valid contact is created with the correct information. I also tested what should happen when the requirements are not met. Lines 28-35 test a null contact ID, and lines 41-48 test an ID that is longer than 10 characters. I used the same approach with the Task and Appointment classes. In `AppointmentTest.java`, I tested a null appointment date and also made sure an appointment could not be created with a date in the past. This helped me make sure my tests were actually connected to the requirements instead of only checking that valid objects could be created.

I also tested each of the service classes. For the Contact and Task services, I tested adding, deleting, and updating objects. I also tested that duplicate IDs were not allowed. For example, `ContactServiceTest.java` uses `assertThrows` when trying to add two contacts with the same ID. I also tested updating the first name, last name, phone number, and address. In `TaskServiceTest.java`, I tested updating both the name and description. For the Appointment service, I tested adding and deleting appointments and made sure duplicate appointment IDs were rejected. Testing each requirement separately made it easier for me to see if something was missing.

I think my JUnit tests were effective because the final project had 97.19% code coverage, which was well above the required 80%. All of my tests were also passing when I submitted the project. The high coverage showed that most of my code was being reached by my tests. At the same time, I learned that having passing tests or high coverage does not automatically mean everything has been tested correctly. My Task milestone was a good example of this. My original tests passed, but I missed some of the validations. After receiving feedback, I added tests for a null task ID, null name, and null description. This helped me understand that I still need to compare my tests to the requirements even when everything is green.

While writing my tests, I tried to keep my code technically sound by making sure every test had a clear purpose and expected result. I used `assertEquals` when I needed to make sure the correct information was stored and updated. I used `assertThrows` when invalid information was supposed to cause an `IllegalArgumentException`. For example, lines 17-20 of `TaskTest.java` use `assertEquals` to check the information in a valid task, while lines 26-31 use `assertThrows` to make sure a task ID over 10 characters is rejected. I used similar tests for null values, descriptions that were too long, duplicate IDs, and appointment dates in the past.

I also tried to keep my tests efficient by making each test focus on one specific requirement. Instead of creating one large test that checked everything at once, I separated the tests into methods such as `testNullTaskId`, `testNullName`, and `testNullDescription`. This made it much easier to understand what went wrong when a test failed. I also used simple valid information when that information was not the focus of the tests. This kept my tests easier to read and prevented me from adding code that was not necessary.

Reflection

Testing Techniques

The main testing I used for this project was unit testing with JUnit 5. This allowed me to test individual classes and methods instead of trying to test the entire application at once. I also used requirement-based testing because I created my tests around the requirements provided by the customer. Another important part of my approach was using both positive and negative tests. My positive tests checked that valid information worked correctly, while my negative tests checked that invalid information was rejected. I also tested boundary values, such as an ID that was longer than the allowed character limit and used exception testing to make sure invalid information caused an `IllegalArgumentException`.

There are other testing techniques that I did not use for this project. One is integration testing, which tests whether different parts of an application work correctly together. I also did not use system testing, which tests the completed application as a whole, or acceptance testing, where the finished software is checked to make sure it meets the customer's needs. Another technique we learned about was mocking, which can create a fake version of something the program depends on, such as a database. Mocking was not needed for this project because the contact, task, and appointment services stored everything in memory and did not use a database.

Each of these techniques can be useful depending on the project. Unit testing was a good fit for Project One because I was working with individual classes and services. Integration testing would be more useful if these services needed to connect to a database or another part of an application. System testing would become important once the entire mobile application was finished because it would allow the whole application to be tested together. Acceptance testing would help determine if the final application actually met what the customer wanted. Mocking

could also be helpful in a larger project if the application depended on a database or outside service that I did not want to use every time i ran a test. Using more than one testing technique would give developers a better idea of how reliable the complete application is.

Testing Mindset

One of the biggest things I learned from this project was that I need to be cautious when testing my own code. At first, I was mostly focused on making sure the code worked when I entered valid information. As I worked through the milestones, I started thinking more about what could go wrong. I tested null values, values that were too long, duplicate IDs, IDs that did not exist, and appointment dates in the past. Being cautious was important because the classes and services worked together. Even if the service worked correctly, there could still be a problem if the object itself accepted information that did not meet the requirements.

I also learned that bias can be a problem when developers test their own code. Since I wrote the code, it was easy to assume that something worked because I already knew what I expected it to do. My Task milestone showed me why this can be a problem. I originally thought my tests were complete because they were passing, but my instructor pointed out that I had not tested all of the validations. I went back and added the missing null tests before submitting Project One. That experience showed me that I cannot rely only on whether my tests are green. I need to go back though the requirements and purposely look for situations I may have missed. In a real project, having another developer review my code and tests could also help find things I overlooked.

Finally, I think being disciplined is important because it can be tempting to skip testing when the code already seems to work. Cutting corners might save time at first, but it can create bigger problems later if defects make it into the final product. It can also create technical debt if

problems or unfinished testing keep getting pushed aside. I plan to avoid this by testing my code as I develop it instead of waiting until the end, fixing problems when I find them, and rerunning my tests whenever I make changes. I also want to continue using the requirements as a checklist, so I know each one has been tested. This project taught me that testing is not just about proving that my code works. I also need to try to find the ways it could fail before the software reaches the user.

References

Southern New Hampshire University. (2026). *CS 320: Software testing, automation, and quality assurance*.

Southern New Hampshire University. (2026). *CS 320 Project Two guidelines and rubric*.